

SECURERISC

A Hardware-Assisted Secure RISC-V-Style Processor

Final Project Report

Prepared by: Mayur S Nagoji

Department of Electronics and Communication Engineering

BMS Institute of Technology and Management (BMSIT&M), Bengaluru

August 2026

Repository: github.com/mayursn-007/SecureRISC

Release: v1.0.0

Implementation: Verilog RTL | Simulation: Icarus Verilog | Waveform Analysis: GTKWave | CI: GitHub Actions

Table of Contents

2. Abstract	3
3. Keywords	3
4. Introduction	3
5. Problem Statement	3
6. Motivation	3
7. Objectives	4
8. Background	4
8.1 RISC-V-Style Processor Architecture	4
8.2 RTL Design	4
8.3 Hardware Security	4
8.4 Memory Protection	4
8.5 Instruction Verification	5
9. System Architecture	5
10. Processor Components	6
10.1–10.9 Component Descriptions	6
11. Security Architecture	7
12. Enhanced Instruction Verification	7
13. Memory Protection Unit	8
14. Security Threat Scenarios	8
14.1 Illegal Instruction	8
14.2 Unauthorized Memory Access	8
14.3 Protected Memory Write	9
15. RTL Implementation	9
16. Verification Methodology	9
17. Testbench Architecture	9
18. Simulation Environment	9
19. GTKWave Waveform Analysis	10
19.1 Instruction Verifier	10
19.2 Memory Protection Unit	11
19.3 CPU Core Integration	11
19.4 Security Regression	12
19.5 Protected Memory Write Attack	13
20. Verification Results	13
21. Security Regression Testing	14
22. Continuous Integration Using GitHub Actions	14
23. Results and Discussion	15
24. Limitations	15
25. Future Work	16
26. Conclusion	16
27. References	16
28. Appendix	17

2. Abstract

SecureRISC is a 32-bit RISC-V-style processor core implemented in Verilog RTL, developed to study how instruction-level and memory-level security policies can be enforced directly in hardware rather than in software running on top of the processor. The design implements a single-cycle datapath supporting a small subset of the RV32I instruction encoding (eight R-type ALU operations, ADDI, LW, SW, BEQ and JAL), combined with two dedicated security blocks: an Enhanced Instruction Verification unit that checks the opcode, funct3 and funct7 fields of every fetched instruction before it reaches the decoder, and a Memory Protection Unit (MPU) that restricts data memory accesses to defined address regions. The processor and its individual sub-modules were verified using Icarus Verilog testbenches, with waveform inspection carried out in GTKWave. A GitHub Actions workflow automatically compiles and runs two of the testbenches (the instruction verifier and the full CPU core) on every push and pull request. Independent re-execution of the repository's testbenches during the preparation of this report confirmed 21 of 21 instruction-verification test cases passing, 7 of 7 memory-protection test cases passing, successful CPU arithmetic and load/store integration, and correct blocking of a simulated unauthorized write to a protected memory region. This report documents the architecture, RTL implementation, verification methodology, and current limitations of the SecureRISC v1.0.0 release.

3. Keywords

RISC-V-style processor, RTL design, Verilog HDL, hardware-assisted security, instruction verification, Memory Protection Unit, Icarus Verilog, GTKWave, functional verification, continuous integration.

4. Introduction

The growing deployment of low-cost embedded and IoT devices has increased the exposure of processor hardware to attacks that target firmware integrity and memory access control. Many small microcontroller-class cores provide little or no built-in mechanism to reject malformed instructions or to restrict which memory regions a running program may read from or write to. SecureRISC is a student RTL project that investigates a lightweight way to add two such mechanisms – instruction verification and memory protection – directly into a simple RISC-V-style processor pipeline, rather than relying on an operating system or a software monitor.

The current release (v1.0.0) implements a single-cycle, non-pipelined core with a small, fixed instruction program held in an on-chip instruction memory. The project is not a full RV32I-compliant core, has not been synthesized for FPGA or ASIC, and has not undergone formal verification; these points are stated explicitly in Section 24 (Limitations) and are reflected consistently throughout this report.

5. Problem Statement

Resource-constrained embedded systems frequently rely on software-based protection mechanisms that consume processing cycles and can be bypassed if the underlying hardware itself is compromised or if malformed instructions reach the execution stage. The problem addressed by this project is the absence, in many small processor cores, of a hardware-level check that (a) validates an instruction's encoding before it is allowed to modify architectural state, and (b) enforces which memory regions a data access may target. SecureRISC explores a lightweight RTL-level solution to both problems within the scope of a small, single-cycle RISC-V-style core.

6. Motivation

Hardware-level security checks execute in parallel with normal instruction flow and cannot be disabled by a compromised software layer running on the same core, which makes them attractive for resource-constrained IoT and embedded applications where a full memory-management unit or a trusted execution environment would be too costly. Implementing instruction verification and memory protection as combinational logic blocks inside the datapath keeps the additional hardware small and lets their behaviour be verified directly through RTL simulation, which is well suited to an academic RTL/VLSI project built with open-source tools.

7. Objectives

- Design a modular, single-cycle RISC-V-style processor datapath in synthesizable Verilog.
- Implement an Enhanced Instruction Verification unit that checks opcode, funct3 and funct7 fields and flags any instruction outside a defined, supported set.
- Implement a hardware Memory Protection Unit that partitions the data address space into code, data, protected and blocked regions.
- Integrate both security blocks into the processor datapath so that invalid instructions and unauthorized memory operations are blocked without requiring software intervention.
- Verify each RTL module individually and the integrated core using Icarus Verilog testbenches, and inspect the resulting behaviour in GTKWave.
- Automate part of the verification flow using a GitHub Actions continuous-integration workflow.

8. Background

8.1 RISC-V-Style Processor Architecture

RISC-V is an open instruction set architecture built around a small base integer instruction set (RV32I) that can be extended modularly. SecureRISC follows the RV32I instruction encoding for the subset of instructions it implements – the eight R-type ALU operations (ADD, SUB, AND, OR, XOR, SLL, SRL, SLT), ADDI, LW, SW, BEQ and JAL – but it does not implement the complete RV32I base set (for example, there is no SUBI-equivalent immediate coverage beyond ADDI, no full set of branch conditions beyond BEQ, and no upper-immediate or system instructions). The project should therefore be described as RISC-V-style, or RV32I-inspired, rather than as a RISC-V-compliant core.

8.2 RTL Design

Register-transfer level (RTL) design describes digital hardware as the flow of data between registers and the combinational logic that operates on that data each clock cycle. SecureRISC is written entirely in structural and behavioural Verilog: each functional unit (program counter, register file, ALU, and so on) is coded as an independent module with a well-defined port list, and the top-level module, `secure_risc_core`, wires these modules together to form the complete datapath.

8.3 Hardware Security

Hardware-level security refers to protection mechanisms implemented as digital logic rather than as software running on top of a processor. Because such mechanisms operate every cycle as part of the datapath, they are active even before an operating system or application software has started, and they cannot be disabled purely from software running on the same core. Instruction verification and memory protection, as implemented in this project, are both examples of hardware security enforced in this way.

8.4 Memory Protection

A memory protection mechanism restricts which addresses a processor may read from or write to, typically by comparing the requested address against a set of permitted ranges. This is a simpler mechanism than a full memory-management unit (MMU), since it does not perform address translation; it only grants or denies access based on fixed address ranges. The MPU implemented in SecureRISC follows this simpler, range-comparison model.

8.5 Instruction Verification

Instruction verification, in the context of this project, refers to checking that a fetched instruction's opcode and function fields correspond to one of a known, supported set of encodings before that instruction is allowed to influence the register file, program counter, or data memory. Any encoding outside this supported set is treated as invalid and is prevented from executing.

9. System Architecture

The SecureRISC core is a single-cycle datapath. On each clock edge the program counter supplies an address to the instruction memory; the fetched instruction is first checked by the instruction verifier and only then passed to the instruction decoder, register file, immediate generator and ALU; the ALU result is used both as the effective address for the memory protection unit and as the address/result path into data memory; and the writeback multiplexer selects between the ALU result, the data memory read value, or PC+4 (for JAL) before writing back to the register file.

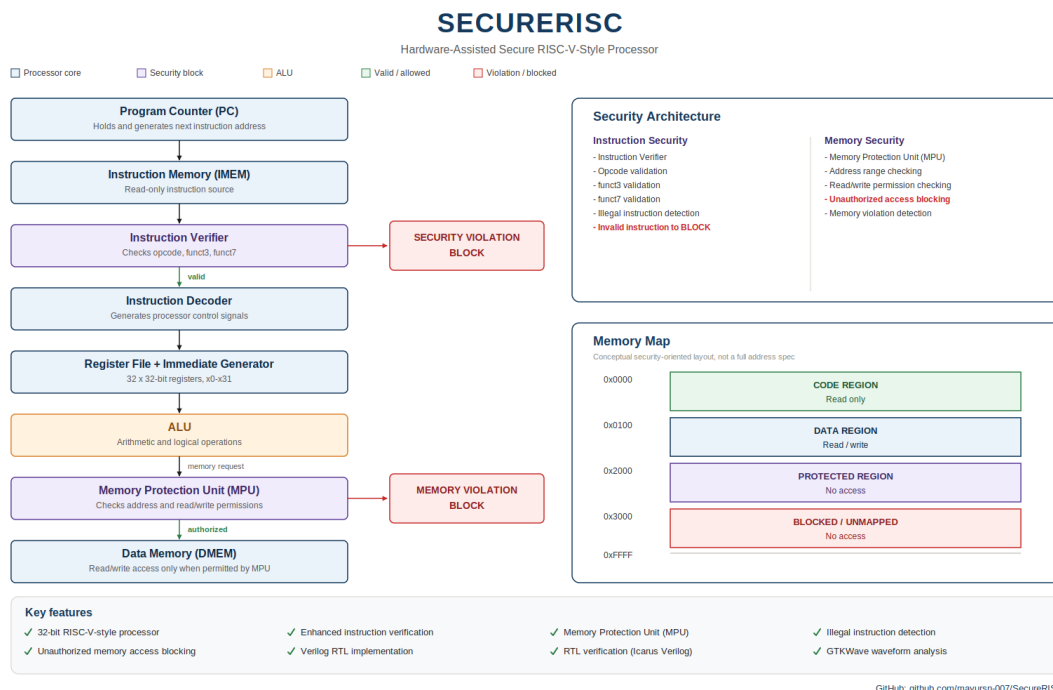


Figure 1. System architecture diagram, as published in the project repository (diagrams/securerisc_architecture.png).

This published diagram groups the Register File and Immediate Generator into a single visual block for compactness. In the actual RTL, these are two separate modules (register_file and immediate_generator) that are instantiated independently in secure_risc_core.v and operate in parallel from the same decoded instruction fields; they are treated as separate blocks throughout the remainder of this report, consistent with the RTL.

The data-flow order implemented in secure_risc_core.v is as follows:

- Program Counter → Instruction Memory → Instruction Verifier
- Instruction Verifier → Instruction Decoder (invalid instructions are substituted with a NOP encoding, 32'h00000013, before reaching the decoder and immediate generator)
- Instruction Decoder → Register File and Immediate Generator (in parallel)
- Register File / Immediate Generator → ALU (ALU second operand is chosen between register data and the immediate value using `alu_src`)
- ALU result → Memory Protection Unit (as the effective data-memory address) and → Data Memory
- Data Memory / ALU result / PC+4 → Writeback multiplexer → Register File write port

10. Processor Components

10.1 Program Counter

Implemented in `rtl/program_counter.v` as a parameterized (`DATA_WIDTH = 32`) synchronous register. On reset it clears to zero; otherwise, if `pc_write` is asserted (by a taken branch or a JAL instruction) it loads `next_pc`, and in all other cases it increments by 4 each clock cycle.

10.2 Instruction Memory

Implemented in `rtl/instruction_memory.v` as a 256-word (32-bit) array with combinational, word-aligned read access (`address[9:2]` selects the word). The memory is not a general loadable program store; it is initialized at elaboration time with a fixed five-instruction program (with the remaining locations filled with the NOP encoding 32'h00000013):

Address	Instruction (hex)	Meaning
0x00	0x00A00093	ADDI x1, x0, 10
0x04	0x01400113	ADDI x2, x0, 20
0x08	0x002081B3	ADD x3, x1, x2
0x0C	0x10302023	SW x3, 0x100(x0)
0x10	0x10002203	LW x4, 0x100(x0)
0x14	0x00000013	ADDI x0, x0, 0 (NOP)

10.3 Instruction Verifier

Implemented in `rtl/instruction_verifier.v` as purely combinational logic. It decodes the opcode (`instruction[6:0]`), `funct3` (`instruction[14:12]`) and `funct7` (`instruction[31:25]`) fields and asserts `instruction_valid` / de-asserts `security_violation` only for the specific encodings listed in Section 12. All other encodings default to `instruction_valid = 0` and `security_violation = 1`.

10.4 Instruction Decoder

Implemented in `rtl/instruction_decoder.v`. It extracts the `rs1`, `rs2`, `rd`, `funct3` and `funct7` fields and generates the control signals `reg_write`, `alu_src`, `mem_read`, `mem_write`, `branch`, `jump` and `alu_control` from the opcode/`funct3`/`funct7` combination. All control signals default to a safe, inactive state before the opcode case statement, so an unrecognised opcode produces no register or memory side effects.

10.5 Register File

Implemented in `rtl/register_file.v` as 32 registers of 32 bits each, with two combinational read ports and one synchronous write port. Register `x0` is forced to zero on every read (via a multiplexer on the read ports) and is also forced to zero unconditionally on every clock edge in the write-logic block. It should be noted that the reset signal in this module only explicitly clears register 0; the general-purpose registers `x1–x31` are not cleared on reset and retain their simulation-default (unknown, 'x') value until first written. This is documented in Section 24 as a limitation rather than presented as a full synchronous-reset register file.

10.6 Immediate Generator

Implemented in `rtl/immediate_generator.v`. It produces sign-extended immediates for the four instruction formats the project supports: I-type (ADDI/LW), S-type (SW), B-type (BEQ) and J-type (JAL). Instruction formats outside this set (for example U-type or full RV32I system encodings) are not handled and fall through to a default immediate of zero.

10.7 ALU

Two ALU implementations exist in the repository. `rtl/alu.v` is an earlier 4-bit-opcode combinational ALU with a single Result/Zero output pair. `rtl/alu_v2.v` is a later, parameterized (`DATA_WIDTH = 32`) version that adds carry, overflow, zero and negative flags and is the version actually instantiated inside `secure_risc_core.v`; `alu.v` is present in the repository (with its own testbench, `alu_tb.v`) but is not part of the integrated core. Both versions support the same eight operations: ADD, SUB, AND, OR, XOR, SLL, SRL and SLT (signed comparison).

10.8 Memory Protection Unit

Implemented in `rtl/memory_protection_unit.v` as combinational address-range comparison logic. It takes the ALU result as the candidate data address together with `mem_read` and `mem_write`, and produces `read_allowed`, `write_allowed` and `memory_violation` according to the fixed region table given in Section 13.

10.9 Data Memory

Implemented in `rtl/data_memory.v` as a 256-word (32-bit) array. Reads are combinational and gated by `mem_read`; writes are synchronous (posedge `clk`) and gated by `mem_write`. In the integrated core, the `mem_read` and `mem_write` inputs to this module are the MPU-gated signals `protected_mem_read` and `protected_mem_write`, not the raw decoder outputs – this is the mechanism by which the MPU actually prevents an unauthorized access from reaching the memory array (see Section 11).

11. Security Architecture

SecureRISC enforces its two security policies at two different points in the datapath. Instruction-level enforcement happens between instruction fetch and decode: the `instruction_verifier` module examines every fetched instruction combinatorially, and `secure_risc_core.v` feeds the decoder and immediate generator with either the original instruction (if `instruction_valid` is asserted) or the fixed NOP encoding `32'h00000013` (if it is not). This means an invalid instruction cannot generate a register write, a memory access, or a branch/jump – it is functionally converted into a no-operation for that cycle, and the program counter simply advances by four as usual.

Memory-level enforcement happens between the ALU and data memory: the `memory_protection_unit` compares the ALU result (used as the data address) against its region table and produces `read_allowed` and `write_allowed`. These are combined with the decoder's `mem_read/mem_write` outputs to form `protected_mem_read` and `protected_mem_write`, and it is these gated signals – not the raw decoder outputs – that are connected to the `data_memory` module's control inputs. An unauthorized access therefore never reaches the memory array; it is blocked at the interface between the ALU/MPU stage and the memory stage.

12. Enhanced Instruction Verification

The instruction verifier accepts the following opcode / funct3 / funct7 combinations; every other combination is rejected (instruction_valid = 0, security_violation = 1):

Instruction	Opcode	funct3	funct7 condition
ADD	0110011	000	0000000
SUB	0110011	000	0100000
SLL	0110011	001	0000000
SLT	0110011	010	0000000
XOR	0110011	100	0000000
SRL	0110011	101	0000000
OR	0110011	110	0000000
AND	0110011	111	0000000
ADDI	0010011	000	–
LW	0000011	010	–
SW	0100011	010	–
BEQ	1100011	000	–
JAL	1101111	–	– (accepted unconditionally)

This is a whitelist, not a general RV32I decoder check: encodings for instructions the project does not implement at all (for example LUI, AUIPC, other branch types, or system instructions) fall through the default case and are treated identically to a genuinely malformed word such as 0xFFFFFFFF or 0x00000000, both of which are used as illegal-instruction test vectors in the repository's testbenches.

13. Memory Protection Unit

The MPU implements the following fixed address-region table, taken directly from the comments and case logic in rtl/memory_protection_unit.v:

Region	Address Range	Read	Write
CODE	0x00000000 – 0x000000FF	Allowed	Blocked
DATA	0x00000100 – 0x00001FFF	Allowed	Allowed
PROTECTED	0x00002000 – 0x00002FFF	Blocked	Blocked
Everything else	–	Blocked	Blocked

A memory_violation is asserted whenever a read is requested to an address where read_allowed is low, or a write is requested to an address where write_allowed is low. This table defines the actual, implemented protection regions used by the RTL and by the MPU testbench; it is a security-region map rather than a complete physical memory map for a full system-on-chip (it does not, for example, define peripheral, stack, or instruction-fetch address ranges outside of what the instruction memory itself uses).

14. Security Threat Scenarios

14.1 Illegal Instruction

Tested at the module level by `testbench/instruction_verifier_tb.v`, which applies 21 directed instruction encodings (13 valid, 8 invalid – covering an invalid R-type funct3, an invalid R-type funct7, an invalid ADDI funct3, an invalid LOAD funct3, an invalid STORE funct3, an invalid BRANCH funct3, an all-zero word, and an all-ones word) and checks `instruction_valid/security_violation` against the expected result for each. Tested again at the core level by `testbench/secure_risc_security_tb.v`, which drives the DUT with a valid ADD instruction followed by the illegal words `0xFFFFFFFF` and `0x00000000`, injected using Verilog's procedural force statement on the internal instruction signal rather than by fetching them from instruction memory.

14.2 Unauthorized Memory Access

Read-side access control is exercised at the module level by `testbench/memory_protection_unit_tb.v`, which includes a directed test of a protected-region read (address `0x2000`) and an out-of-range read (address `0x4000`), both of which correctly report `read_allowed = 0` and `memory_violation = 1`. The repository does not include a dedicated core-level (instruction-driven) unauthorized-read attack scenario; only the write path is exercised at the full-core level, as described in 14.3.

14.3 Protected Memory Write

Exercised at the core level by `testbench/memory_attack_tb.v`. This testbench does not construct a native SW instruction targeting the protected region; instead, after reset it uses force to directly override the internal `DUT.alu_result` signal to `0x00002000` and the internal `DUT.mem_write` signal to 1, simulating the effect an attacking instruction would have on the datapath at the point where the MPU makes its decision. The testbench then checks that `write_allowed = 0`, `memory_violation = 1` and `protected_mem_write = 0`, confirming that the write does not reach data memory. This distinction – signal-level fault injection rather than an end-to-end malicious instruction fetched from instruction memory – is noted here because it affects how strongly this test can be read as evidence of protection against a real instruction-stream attack, and is discussed further in Section 24.

15. RTL Implementation

The design is written in Verilog and organised as one top-level structural module, `secure_risc_core.v`, instantiating nine leaf modules: `program_counter`, `instruction_memory`, `instruction_verifier`, `instruction_decoder`, `register_file`, `immediate_generator`, `alu_v2`, `memory_protection_unit` and `data_memory`. Parameterization (`DATA_WIDTH`, and `ADDR_WIDTH` where applicable) is used in `program_counter`, `register_file`, `alu_v2` and `data_memory`. The repository's CI workflow compiles the design with Icarus Verilog using the `-g2012` flag, which enables broader language-feature support in the simulator; the RTL itself is written mainly in a Verilog-2001-compatible style, with no vendor-specific primitives, generate blocks, or SystemVerilog interfaces used. No synthesis run, gate-level netlist, or timing report is currently part of the repository.

16. Verification Methodology

The thirteen testbenches in the repository fall into two categories. Five are self-checking: they contain explicit if/else comparisons against expected values and print an explicit PASS or FAIL status line per test – `instruction_verifier_tb.v` (21 cases), `memory_protection_unit_tb.v` (7 cases), `secure_risc_core_tb.v` (1 integration test), `secure_risc_security_tb.v` (3 cases) and `memory_attack_tb.v` (1 case). The remaining eight – `alu_tb.v`, `alu_tb_v2.v`, `register_file_tb.v`, `program_counter_tb.v`, `instruction_decoder_tb.v`,

immediate_generator_tb.v, instruction_memory_tb.v and data_memory_tb.v – are directed testbenches that apply stimulus and print signal values with `$display`, without an automated pass/fail comparison; verifying these requires visually comparing the printed values against the expected result for each operation. This distinction is preserved in the results table in Section 20.

17. Testbench Architecture

Each testbench instantiates its device under test directly (either a single leaf module, or the full `secure_risc_core` for integration-level tests), generates a clock with an always `#5 clk = ~clk` block where a clock is required, and applies stimulus either by direct assignment to input registers (for standalone module tests) or, for the two attack-style core-level tests (`secure_risc_security_tb.v` and `memory_attack_tb.v`), by using Verilog's `force/release` procedural statements to override selected internal DUT signals (`instruction`, `alu_result`, `mem_write`, `mem_read`) and then restoring them afterward. Every testbench opens a VCD dump file via `$dumpfile/$dumpvars` for later inspection in GTKWave, and terminates with `$finish`.

18. Simulation Environment

Simulation is performed with Icarus Verilog (`iverilog` to compile, `vvp` to execute); the badges and workflow in the repository specify Icarus Verilog as the simulator and GTKWave as the waveform viewer. For this report, the repository was cloned and every testbench was independently recompiled and re-run using Icarus Verilog 12.0 to confirm the results quoted in Sections 12, 13, 20 and 21, rather than relying only on figures supplied in the task description.

19. GTKWave Waveform Analysis

19.1 Instruction Verifier

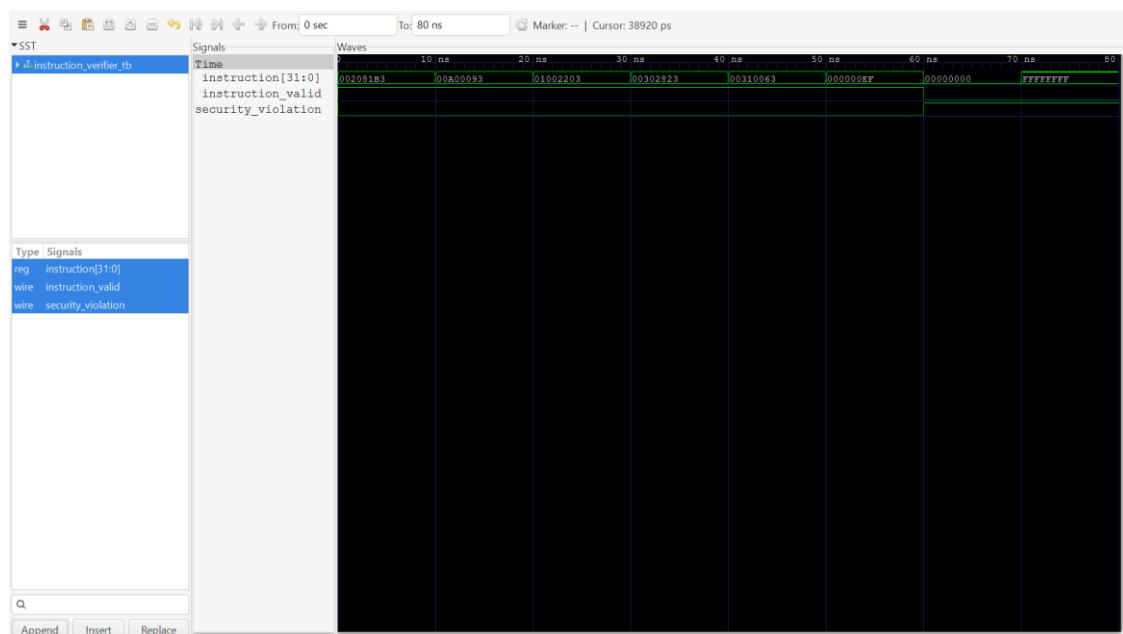


Figure 2. Instruction verifier testbench waveform: `instruction`, `instruction_valid` and `security_violation` across the 21 directed test cases.

The instruction bus carries each of the 21 directed test vectors in sequence (for example 002081B3 for ADD, followed by an ADDI, an SW, a BEQ, a JAL, and finally the illegal values 00000000 and FFFFFFFF). Before each transition, `instruction_valid` and `security_violation` hold the result of the previous test case. During the interval in which a given instruction value is present, the two outputs settle combinationaly to their result for that

opcode/funct3/funct7 combination; for the final two vectors (all-zero and all-one words, which do not match any accepted opcode) `instruction_valid` drops to 0 and `security_violation` rises to 1, visible as the outputs staying at their blocked state for the remainder of the trace. After each test case, the testbench moves to the next vector and the outputs update again on the next combinational evaluation. The waveform demonstrates the expected behaviour because every accepted encoding produces `instruction_valid = 1 / security_violation = 0` for its entire display interval, and every rejected encoding produces the opposite, with no intermediate or glitching values visible in the traces at this timescale.

19.2 Memory Protection Unit

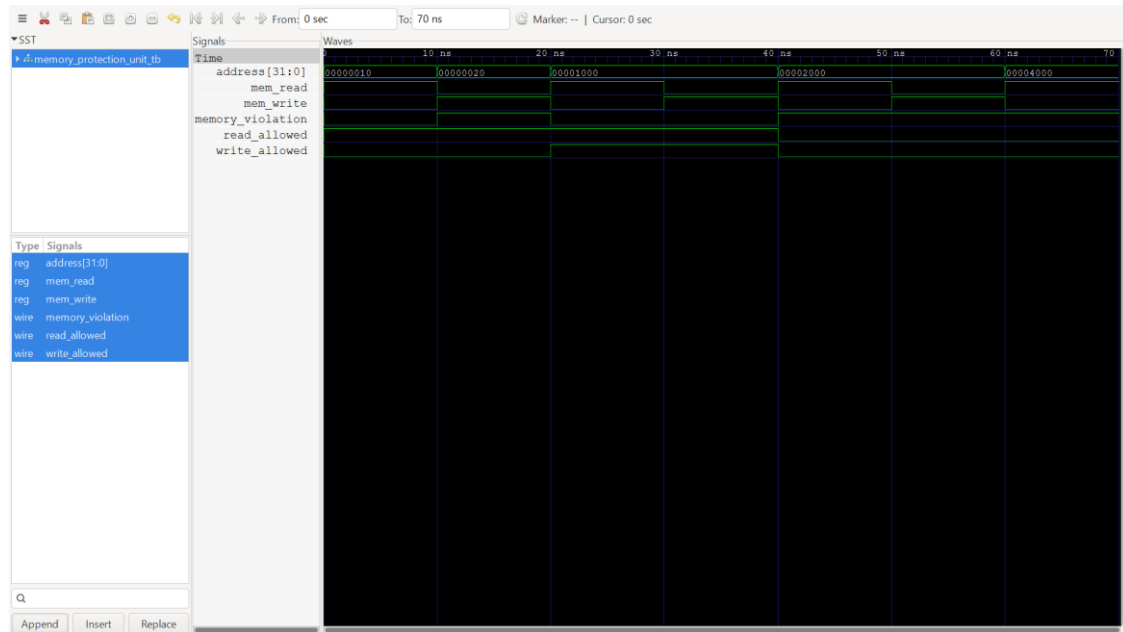


Figure 3. MPU testbench waveform: address, `mem_read`/`mem_write`, `memory_violation`, `read_allowed` and `write_allowed` across the seven directed test cases.

The address bus steps through the seven MPU test addresses (0x10 and 0x20 in the CODE region, 0x1000 in the DATA region twice, 0x2000 in the PROTECTED region twice, and 0x4000 outside all defined regions). Before each address change, `read_allowed`, `write_allowed` and `memory_violation` reflect the previous test's region. During each interval, the combinational region-decode logic settles: `read_allowed` and `write_allowed` both go high only for the DATA-region address; `read_allowed` alone is high for the CODE-region reads; and both remain low for the PROTECTED and out-of-range addresses. `memory_violation` rises whenever the requested `mem_read` or `mem_write` is not matched by the corresponding allowed signal, which is visible at the CODE-region write test, both PROTECTED-region tests, and the out-of-range test. After each test interval the testbench advances to the next address and the outputs re-settle. This demonstrates that the MPU's combinational logic produces the address-range decisions defined in Section 13 for every test address applied.

19.3 CPU Core Integration

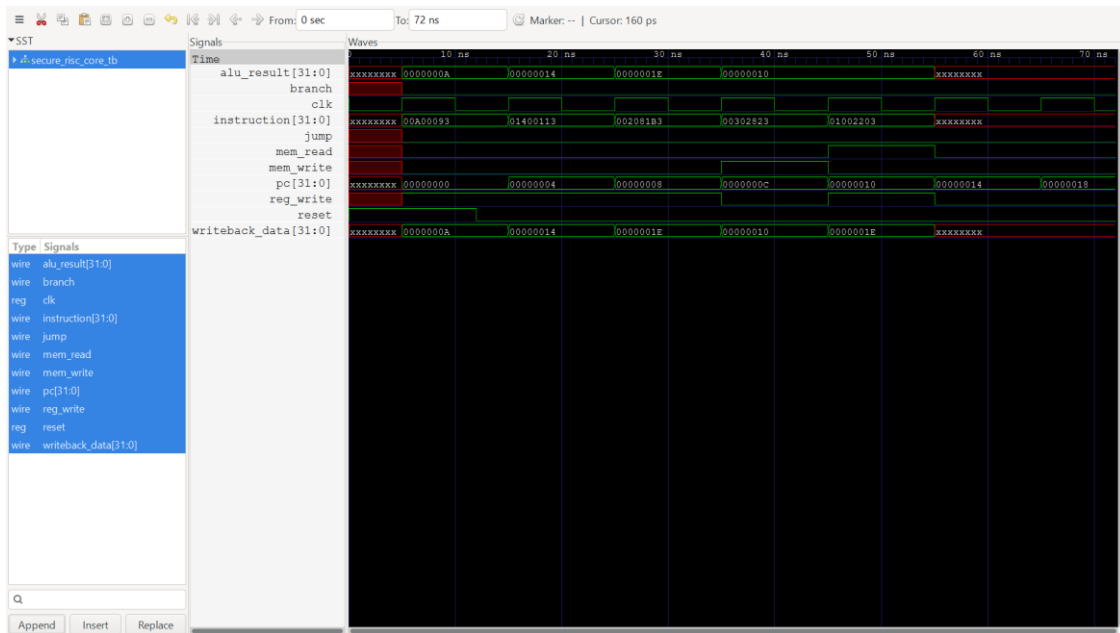


Figure 4. *secure_risc_core_tb* waveform: *pc*, *instruction*, *alu_result* and *writeback_data* over the five-instruction program.

The *pc* bus increments by 4 each cycle (0x00, 0x04, 0x08, 0x0C, 0x10, 0x14), fetching the ADDI x1,10, ADDI x2,20, ADD x3,x1,x2, SW x3,0x100(x0) and LW x4,0x100(x0) instructions in order, matching the fixed program in instruction memory (Section 10.2). Before each instruction executes, *alu_result* and *writeback_data* hold the previous instruction's result. During each cycle, *alu_result* reflects the current ALU operation (0x0A for the first ADDI, 0x14 for the second, 0x1E for the ADD, and the computed store/load address 0x100 for the SW/LW pair), and *writeback_data* reflects what is written back to the register file for that instruction (the ALU result for the ADDI/ADD instructions, and the value read back from data memory for the LW). After the fifth instruction, the trace shows the register values settling to their final state (x1=10, x2=20, x3=30, x4=30, as also reported in the \$display output). This demonstrates that arithmetic (ADDI, ADD) and the store-then-load round trip through data memory both function as intended for this fixed instruction sequence.

19.4 Security Regression

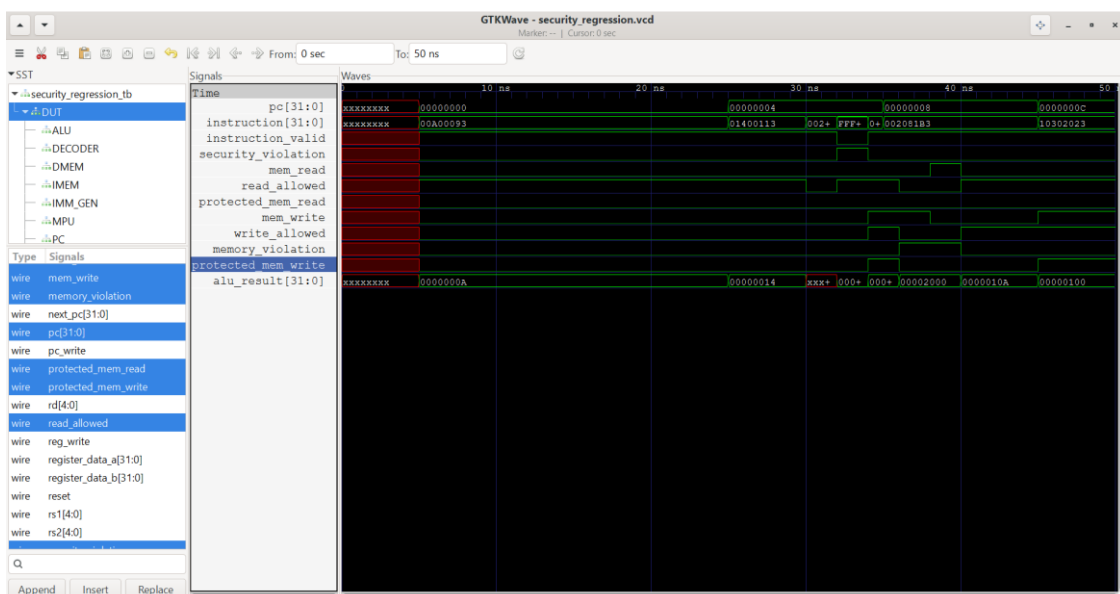


Figure 5. *secure_risc_security_tb* waveform (GTKWave application window): *instruction*, *instruction_valid*, *security_violation*, *mem_write*, *memory_violation* and *protected_mem_write*.

This trace covers the three-test *secure_risc_security_tb* sequence layered on top of the normal program execution. Before the forced instruction injections, the DUT is running the fixed program from instruction

memory (visible as the pc and instruction values matching Section 10.2). During the injection windows, the instruction signal is overridden first with a valid ADD encoding (instruction_valid stays 1, security_violation stays 0, confirming the override does not itself break normal acceptance), then with 0xFFFFFFFF and 0x00000000 in turn (instruction_valid drops to 0 and security_violation rises to 1 for the duration of each injected word). After each injected value is released, the trace returns to reflect the underlying program instruction stream and pc continues advancing. The waveform demonstrates that the security-violation output tracks the injected instruction combinationally and correctly distinguishes the one legal injected encoding from the two illegal ones.

19.5 Protected Memory Write Attack

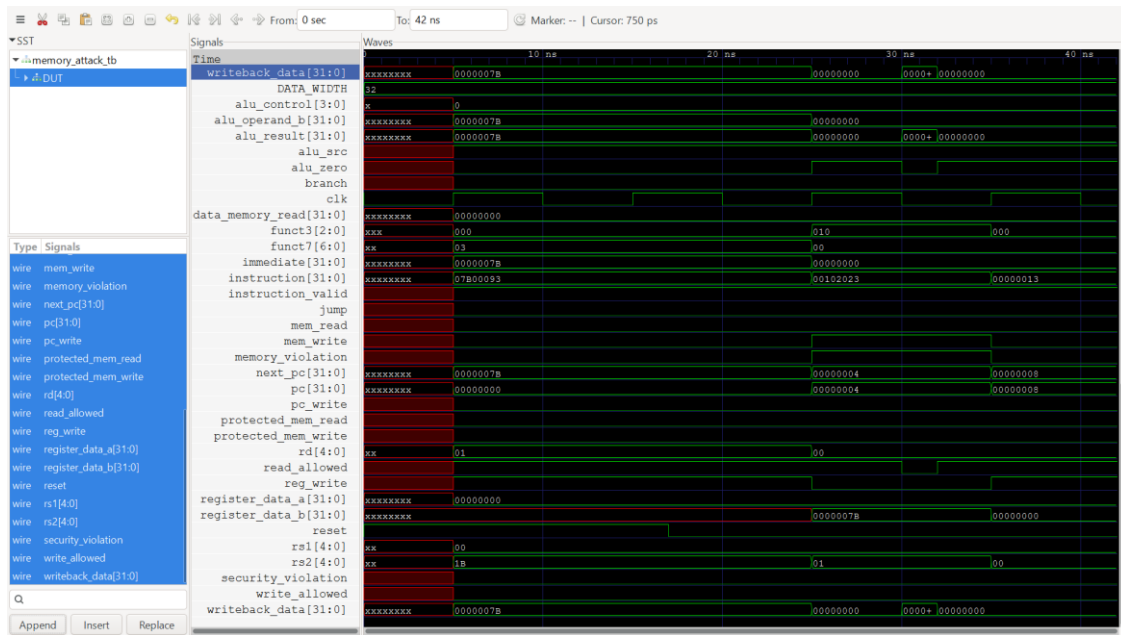


Figure 6. `memory_attack_tb` waveform: `mem_write`, `memory_violation`, `write_allowed`, `protected_mem_write` and `writeback_data` around the forced write attempt.

Before the attack window, the core is executing the two ADDI instructions from the fixed program in `memory_attack_tb.v` (loading x1 with 123), with `mem_write` low and `memory_violation` low. At the attack point, the testbench forces DUT.alu_result to 0x00002000 (the PROTECTED region) and DUT.mem_write to 1. During this forced interval, `write_allowed` is observed to be 0 (the MPU does not grant write permission for this address), `memory_violation` rises to 1, and `protected_mem_write` – the signal actually connected to the data_memory write-enable input – remains 0. After the forced signals are released, `mem_write` and `memory_violation` return to the values driven by the normal decoder/MPU logic for the ongoing instruction stream. The waveform demonstrates that even when an internal control signal is forced to request a write to the protected region, the gating logic between the MPU and data memory (`protected_mem_write`) prevents that write from reaching the memory array.

20. Verification Results

Results below were reproduced by independently recompiling and re-running every testbench in the repository with Icarus Verilog 12.0 during the preparation of this report.

Test	Purpose	Expected Result	Observed Result	Status
instruction_verifier_tb	Validate opcode/funct3/funct7 filtering (21 directed cases)	21/21 pass	21/21 pass	PASS

Test	Purpose	Expected Result	Observed Result	Status
memory_protection_unit_tb	Validate CODE/DATA/PROTECTED/out-of-range region permissions (7 cases)	7/7 pass	7/7 pass	PASS
secure_risc_core_tb	CPU arithmetic and store/load integration	x1=10, x2=20, x3=30, x4=30	x1=10, x2=20, x3=30, x4=30	PASS
secure_risc_security_tb	Reject illegal instructions injected via force (3 cases)	1 accepted, 2 blocked	1 accepted, 2 blocked	PASS
memory_attack_tb	Block forced write to protected region (0x2000)	write_allowed=0, violation=1	write_allowed=0, violation=1	PASS
alu_tb_v2 (ALU v2)	Directed check of 8 ALU operations	Correct results	Correct (visual check, no auto-assert)	PASS (manual)
register_file_tb / program_counter_tb / instruction_decoder_tb / immediate_generator_tb / instruction_memory_tb / data_memory_tb	Directed unit sanity checks	Correct signal values	Correct (visual check, no auto-assert)	PASS (manual)
GitHub Actions workflow (verilog.yml)	Automated CI: compiles and runs secure_risc_core_tb and instruction_verifier_tb	Both jobs pass	Both jobs pass (per repository badge)	PASS

“Manual” status indicates that the corresponding testbench has no embedded pass/fail assertion and its output was confirmed by comparing the printed \$display values against the expected arithmetic/logic results by hand, not by an automated check inside the testbench itself.

21. Security Regression Testing

testbench/secure_risc_security_tb.v provides the project's dedicated security regression test. After reset, it forces DUT.instruction to a valid ADD encoding (0x002081B3) and confirms instruction_valid = 1 / security_violation = 0; it then forces DUT.instruction to 0xFFFFFFFF and confirms instruction_valid = 0 / security_violation = 1; and finally forces DUT.instruction to 0x00000000 and confirms the same blocked result. All three checks passed on re-execution. This test complements, but does not replace, the broader 21-case sweep in instruction_verifier_tb.v: it exercises the security path inside the full integrated core (rather than the standalone verifier module) but with only three instruction values, and using signal forcing rather than instructions actually fetched from instruction memory.

22. Continuous Integration Using GitHub Actions

The repository defines a single GitHub Actions workflow, .github/workflows/verilog.yml (“SecureRISC CI”), triggered on every push and pull request to the main branch. The workflow runs on an ubuntu-latest runner and performs the following steps: (1) check out the repository; (2) install Icarus Verilog via apt-get; (3) compile the full CPU core (secure_risc_core.v together with all nine leaf RTL modules and testbench/secure_risc_core_tb.v) with iverilog -g2012 and run it with vvp; (4) separately compile rtl/instruction_verifier.v with testbench/instruction_verifier_tb.v and run it with vvp. The workflow badge shown in the repository README reflects the pass/fail state of these two compilation-and-simulation jobs.

This should be described as a continuous integration workflow for RTL regression checking, not as a production CI/CD pipeline – it does not perform synthesis, linting, coverage collection, or deployment of any kind. Its value for an RTL project of this size is that it automatically catches compilation errors, elaboration failures, and regressions in the CPU-integration and instruction-verification behaviour on every commit, without requiring the developer to manually re-run those two testbenches locally. It currently does not automate the memory-protection-unit test, the security-regression test, the memory-attack test, or any of the eight standalone unit testbenches listed in Section 20; those must currently be run manually, as was done for this report.

23. Results and Discussion

The verification activity carried out for this report supports three specific claims about the current release. First, the instruction verifier correctly separates the 13 supported encodings from a representative set of 8 malformed or unsupported encodings, with no false accepts or false rejects observed across the 21 directed cases. Second, the memory protection unit correctly enforces the three-region access table for both in-region and out-of-range addresses, and correctly asserts `memory_violation` only when a disallowed access type is requested. Third, the integrated core executes its fixed five-instruction program correctly, including a store to the DATA region followed by a load of the same value, and the datapath's MPU-gating mechanism (`protected_mem_read` / `protected_mem_write`) prevents a forced write to the PROTECTED region from reaching data memory.

The results should be read alongside two scope limitations already noted above: the attack-style tests (`secure_risc_security_tb`, `memory_attack_tb`) rely on Verilog `force/release` to inject illegal values directly onto internal signals rather than demonstrating the attack purely through an instruction fetched from instruction memory, and the automated CI currently covers only two of the thirteen testbenches. Neither limitation invalidates the results obtained – the underlying combinational logic being tested is the same logic that would process a genuinely fetched malicious instruction or address – but they do mean the current evidence base is narrower than a full instruction-stream-driven penetration test would provide.

24. Limitations

- Instruction set coverage is a small, fixed subset of RV32I (8 R-type ALU ops, ADDI, LW, SW, BEQ, JAL); the project is not RV32I-compliant and does not implement CSR instructions, exceptions, interrupts, or privilege levels.
- The datapath is single-cycle and non-pipelined; there is no pipelining, hazard detection, or branch prediction.
- Instruction memory holds a fixed, hardcoded five-instruction program set at elaboration time; there is no mechanism to load an external program (for example from a hex file) in the current RTL.
- The register file's reset logic only explicitly clears register `x0`; general-purpose registers `x1–x31` are not cleared on reset.
- The memory-protection region table (CODE / DATA / PROTECTED / blocked) is a security-oriented address map, not a complete physical memory map for a full SoC (no peripheral or stack regions are defined).
- The core-level attack testbenches (`secure_risc_security_tb.v`, `memory_attack_tb.v`) use Verilog `force/release` to override internal signals directly, rather than driving the attack purely through instructions fetched from instruction memory; this is a valid RTL verification technique but is narrower in scope than a full instruction-stream-based penetration test.

- Unauthorized-read attacks are exercised only at the standalone MPU module level (`memory_protection_unit_tb.v`); there is no dedicated core-level unauthorized-read attack test, unlike the write path.
- GitHub Actions CI currently automates only 2 of the 13 testbenches in the repository (`secure_risc_core_tb` and `instruction_verifier_tb`); the remaining testbenches, including the MPU, security-regression, and memory-attack tests, must be run manually.
- Eight of the thirteen testbenches (including both ALU testbenches and all remaining standalone unit tests) are directed, display-only testbenches without embedded pass/fail assertions, so their results were confirmed by manual inspection rather than by an automated check.
- No FPGA implementation, synthesis run, timing analysis, or formal verification has been performed on this design; the `docs/` folder in the repository contains planning documents (for example describing UART security, privilege checking, an exception handler, and pipelining) that describe intended future direction rather than features implemented in the current RTL.
- No claim of complete or provable security is made; the project demonstrates that two specific, narrowly-scoped attack patterns are blocked in simulation, not that the design is secure against all classes of hardware attack.

25. Future Work

The following extensions are planned or proposed but not implemented in the current (v1.0.0) release:

- FPGA implementation and on-board validation.
- More complete RISC-V ISA (RV32I) support, including remaining branch types and upper-immediate instructions.
- Exception handling for illegal instructions and memory violations (currently, invalid instructions are silently converted to a NOP rather than trapped).
- Privilege levels (user/supervisor separation).
- Secure boot support.
- Instruction authentication mechanisms beyond opcode/funct field whitelisting.
- Formal verification of the instruction verifier and MPU logic.
- A hardware cryptographic accelerator.
- Cache protection mechanisms (the current design has no cache).
- Bus-level access control for a multi-master or multi-peripheral system.
- Expanded regression coverage, including an instruction-stream-driven (rather than force-injected) core-level attack test suite, and full CI automation of all existing testbenches.

26. Conclusion

SecureRISC v1.0.0 demonstrates, at the RTL simulation level, that a small single-cycle RISC-V-style processor can enforce a whitelist-based instruction check and a region-based memory access check directly within its datapath, without software involvement. Independent re-execution of the repository's testbenches confirmed correct behaviour of the instruction verifier (21/21), the memory protection unit (7/7), the integrated CPU core, and the two targeted security-attack testbenches. The design remains a small, single-cycle, non-pipelined core with a fixed instruction program and a limited instruction set, and several verification and implementation steps – including FPGA realization, formal verification, and broader instruction-stream-driven attack testing – remain as future work. Within this scope, the project provides a concrete, verifiable example of hardware-assisted instruction and memory security suitable for further extension in subsequent coursework or research.

27. References

- Mayur S Nagoji, SecureRISC repository, GitHub, github.com/mayurn-007/SecureRISC (accessed August 2026).
- RISC-V International, “The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture,” riscv.org.
- Icarus Verilog project documentation, steveicarus.github.io/iverilog.
- GTKWave documentation, gtkwave.sourceforge.net.
- GitHub Actions documentation, docs.github.com/actions.

28. Appendix

A.1 Repository File Structure

- rtl/ – alu.v, alu_v2.v, data_memory.v, immediate_generator.v, instruction_decoder.v, instruction_memory.v, instruction_verifier.v, memory_protection_unit.v, program_counter.v, register_file.v, secure_risc_core.v
- testbench/ – one testbench per RTL module plus secure_risc_core_tb.v, secure_risc_security_tb.v and memory_attack_tb.v (13 files total)
- simulation/ – GTKWave screenshots referenced as figures in Section 19
- diagrams/ – securerisc_architecture.png (Figure 1) and its README
- docs/ – planning-stage markdown notes (Design_Methodology, Future_Work, Problem_Statement, Security_Features, System_Architecture)
- reports/ – ALU_Verification.md (a template report, largely marked “Pending” in the repository at the time of writing) and a reports README
- .github/workflows/verilog.yml – the CI workflow described in Section 22
- LICENSE – MIT License, © 2026 Mayur S Nagoji

A.2 Repository and Release Information

Repository: github.com/mayurn-007/SecureRISC

Tagged release referenced in this report: v1.0.0

License: MIT